

Joe Bylund

joseph.bylund@gmail.com • 347-829-5863 • Boston, MA

EXPERIENCE

Klaviyo

Boston, MA

Lead Software Engineer

August 2023 — Present

- **Purchase containment filtering** — designed a bloom filter architecture replacing repeated large-scale ClickHouse scans, bulk-streaming (person, item) pairs into Redis. Reduced data scanned by ~1000x on a path accounting for ~1/3 of total ClickHouse cluster load. Sketched the design for a partner team and shepherded them through implementation (ClickHouse, Redis, python).
- **Information schema as a service** — prototyped a SQL interface over multi-tenant, schema-on-read JSON data, and later rejoined the team that productionized it. Built the manifest service translating user-supplied SQL into queries against JSON-backed tables, giving customers a typed SQL API over untyped storage. Cut manifest resolution latency 3x end to end (python, gRPC, MySQL, StarRocks).
- **Query service** — founding engineer on a gRPC query service over multi-tenant event data: interceptor stack (rate limiting, validation, concurrency limiting, slow-request logging), sharded connection pooling, executors for ClickHouse, PostgreSQL and MySQL, and query rewriting (python, gRPC, ClickHouse, PostgreSQL, MySQL).
- **Metrics service performance** — added profiling instrumentation, then optimized query mapping and result processing, reducing latency ~58% across every request the service serves (python, gRPC, ClickHouse).
- **Custom objects** — built gRPC APIs letting customers define schemas for their own objects, with per-company partitioned rate limiting and a shared Redis Sentinel client, modeled as data vault hubs, links and satellites (python, gRPC, Redis, Pulsar).
- **Developer velocity** — parallelized test containers, added CI cache targets, and built local development environments used across teams.

Altana

New York City (Remote)

Senior Software Engineer

March 2022 — June 2023

- **Business network discovery and traversal** — built tools to construct business networks from graph of world wide shipment data.
- Built search microservice for companies, facilities and transactions using FastAPI, Pydantic, ArangoDB.
- **Arango client library** — built http client for ArangoDB supporting synchronous and asynchronous requests used in search microservice.
- **Spark client library** — shimmed three different client libraries (pyspark, pyhive, databricks-sql-connector) to pep-249 interface, supports creation of business networks.
- **Geocoding client** — writing api client, measuring and improving accuracy, building pipeline.
- **Geocoding pipeline** — built continuous address geocoding pipeline and geocoded ~400 million addresses.

Kensho

Cambridge, MA

Data Engineer

2018 — March 2022

- Designed and implemented multi-step document processing pipeline using multiple in-house ML services.
- Designed and built speech-to-text alignment pipeline using SQS & gentle forced aligner.
- Implemented a number of checks in github hooks (pylint, flake8, mypy...), improving the developer experience.
- Migrated fuzzy company identification service to kubernetes and optimized performance.

Moat

New York, NY

Senior Data Scientist & Back-end Engineer

2013 – 2018

- Designed and implemented distributed, fault-tolerant ETL, reducing cost by an order of magnitude, increasing reliability, and reducing processing time from ~10 hours to ~1 hour, making data available to clients far earlier in the day (using python, SQS, Redis, PostgreSQL).
- **API performance** – cut API latency 3–4x and raised the maximum request size, guided by profiling data (PHP, CakePHP).
- Contributed improvements to ORM (CakePHP) and core PHP in order to reduce the number of queries necessary to render a page by 5x (decreasing page load time by ~3x) (CakePHP, c).
- Standardized deployment framework used to deploy thousands of servers of ~30 different roles to AWS (AWS, EC2, boto3).
- Migrated primary non-statistical database (users, accounts, metadata) from MySQL to PostgreSQL, improving uptime & flexibility (MySQL, PostgreSQL, foreign data wrappers).
- Migrated primary statistical database (500 million rows/day) from non-first normal form to first normal form schema, improving query latency, reducing storage demands, and increasing throughput.
- Architected and implemented sophisticated message routing system which is responsible for moving ~40 billion events per day from pixel servers to real-time processing servers while balancing CPU and memory constraints (c++).
- Architected and prototyped massively parallel decentralized data lake using AWS lambda and S3 for cost-effective storage and low-latency and cost-effective queries (AWS lambda, python, PostgreSQL).

PROJECTS

- [Sylvan Librarian](#) ([source](#)) – open source Magic: The Gathering card search engine in rust, implementing Scryfall's query syntax and extending it with arithmetic expressions. Self-hosted with blue/green deploys behind nginx; independently forked and deployed to Cloudflare Workers by an outside contributor (rust, python).
- [pg_mimic](#) – pure-python asyncio implementation of the PostgreSQL wire protocol, letting any python process present a postgres interface to standard clients rather than a bespoke http api. Verified against psycopg, asyncpg, pg8000 and psql (python, asyncio).

EDUCATION

Columbia University

New York, NY

PhD – Integrated Program In Cellular, Molecular and Biomedical Studies

2007 – 2013

Thesis: Monte-Carlo Sampling of Protein-Ligand Interactions and Computational Improvements to Implicit Solvent Models

- Led development and maintenance of Protein Local Optimization Program (PLOP) project, a molecular mechanics library for protein structure prediction (fortran).
- Designed and implemented the computational mutation scanning module of PLOP.
- Redesigned build system to automatically determine dependencies and take advantage of parallel compilation, reducing build time from ~30 minutes to ~3 minutes and accelerating development.
- Created a small molecule database representing 95%+ of small molecules in the Protein Data Bank, extending PLOP from a protein-only program to a general molecular mechanics toolkit.
- Designed and implemented a Perl based automated regression testing framework, which accelerated

development while minimizing bugs and regressions.

- Created a project wiki, combining scattered documentation and completing missing documentation.

Rice University

Houston, TX

Bachelor of Arts – Mathematics

2003 – 2007, GPA 3.7/4.0

Bachelor of Science – Ecology and Evolutionary Biology

Relevant coursework: Machine Learning, Ordinary and Partial Differential Equations, Real and Complex Analysis, Combinatorics, Number Theory, Mathematical Logic, Modern Algebra, Euclidean and Non-Euclidean Geometry.

- Designed and implemented two methods of combining information from multiple protein crystal structures into a single “composite motif”. These combined motifs increased sensitivity and specificity of motif matching algorithms.
- Completed senior thesis project identifying homologous pseudogenes in human and chimpanzee, and determining differential mutation rates.

OPEN SOURCE CONTRIBUTIONS

Python Package Installer – Pip

- Up to 10x improved performance of package version resolution ([pull request](#)).
- 2x performance increase in comparison of tag objects ([pull request](#)).

These changes decreased the time spent on pip steps as part of ci process (at Kensho).

PHP

Avoided roundtrip to database in order to get column type for most common datatypes ([pull request](#)). This decreased the number of queries run as well as page load time by more than an order of magnitude in some cases (at Moat).

Gnome Shotwell Photo Manager

47 commits from 2013 to 2021, across the import pipeline, thumbnail cache, saved searches, publishing plugins and build system (vala).

- **Import throughput** – reworked thumbnail generation to decode each photo once and derive every thumbnail size from that single decode, rather than decoding once per size ([commit](#)).
- **RAW metadata** – cached metadata on the RAW reader so a file is parsed once per import instead of on every access ([commit](#)).

TECHNOLOGIES & SKILLS

- **Languages** – Python, SQL, rust, C++
- **Data stores** – ClickHouse, PostgreSQL, MySQL, StarRocks, Redis, ArangoDB; previously Vertica, Redshift
- **Platform** – AWS (EC2, S3, RDS, Lambda, DynamoDB), Kubernetes, gRPC; Kafka, Pulsar, SQS, Kinesis, RabbitMQ

Last updated August 2026